BANKING & FINANCIAL INVOICE PROCESSING USING AGENTIC AI

A Rule-Based Multi-Agent Simulation

Project Report

Technology: Python 3 and Streamlit
Application Area: Banking and Financial Operations
Project Type: College Project / Prototype

Swayam Hull - 25070126519
Tushar Chitlangia - 24070126191

Abstract

Invoice verification is an important activity in finance teams because an invoice must be checked against its own details, previous invoices and payment information before it can be accepted for further processing. When these checks are performed manually across separate records, the work can become slow and inconsistent. Small errors in totals, repeated invoice numbers or differences between billed and paid amounts may lead to delayed vendor settlements, unnecessary rework or financial loss.

This project presents a Banking and Financial Invoice Processing prototype built as a rule-based multi-agent simulation. It uses Python functions to represent four specialised agents: Planner, Document, Validation and Critic. An orchestrator coordinates their work. The Planner prepares a five-step task plan, the Document agent reads six structured invoice fields, the Validation agent performs five checks, and the Critic determines the final status from the results. A Streamlit dashboard allows a user to choose a sample invoice and inspect the decision, individual checks, reconciliation details and audit log.

The prototype distinguishes three outcomes: VERIFIED, FLAGGED and INCOMPLETE. VERIFIED means all five checks pass and the invoice is eligible for human review; it does not authorise payment. FLAGGED indicates that a rule has failed, while INCOMPLETE indicates that required information is missing or unusable. The system uses exact Decimal arithmetic for monetary calculations and synthetic in-memory data for demonstration. Nine tests are reported as passing, including tests for the required business scenarios and invalid or empty input.

The main contribution is an explainable workflow in which each agent has a defined responsibility and each outcome is accompanied by reasons. The current version does not use a language model, optical character recognition, a persistent database or real banking connections. These are possible extensions rather than capabilities of the present prototype.

Keywords

Agentic AI, invoice processing, financial reconciliation, rule-based automation, multi-agent workflow, Streamlit, audit log, human-in-the-loop.

Report Structure

The report introduces the problem and objectives, reviews the proposed design, describes the architecture and processing stages, explains the validation logic, presents the sample demonstrations and testing approach, and closes with limitations and future development opportunities.

1. Introduction

1.1 Background

Invoices are used to record amounts that a customer or organisation is expected to pay for supplied goods or services. In a banking or business-finance setting, an invoice may need to be compared with payment records and historical documents before it can be treated as reconciled. The task is not limited to reading the invoice: the values must also be complete, internally consistent and supported by the available records.

Manual verification often requires an employee to switch between invoice details, earlier invoice records and payment information. Repeating this process for many documents can consume time and makes the result dependent on careful human checking. The proposed prototype demonstrates how a clearly defined software workflow can carry out a set of routine checks in a consistent order while leaving the final approval with a person.

1.2 Problem Statement

The project addresses the challenge of verifying structured invoice information through several separate checks. The system must identify missing information, validate the relationship between subtotal, tax and total, detect a repeated invoice number, confirm that a payment record exists, and compare the recorded payment amount with the invoice total. The result should be easy to understand and should explain why an invoice has received a particular status.

1.3 Aim

The aim is to develop a small, transparent invoice-processing simulation that coordinates specialised agents to evaluate invoice data and present an explainable result through a browser-based dashboard.

1.4 Objectives

- Divide the verification workflow into four agents with distinct responsibilities.
- Coordinate agent execution through a central orchestration function.
- Implement five deterministic checks for completeness, arithmetic, duplication and payment reconciliation.
- Produce one final status with supporting reasons, check results and an ordered audit trail.
- Provide sample cases that demonstrate successful, failed and incomplete processing.
- Keep the decision-support process under human control rather than initiating payment automatically.

1.5 Scope

The project is a proof of concept using six synthetic invoices and in-memory records. It demonstrates decision logic and interface behaviour, not a production banking system. It does not extract information from scanned documents, learn from historical examples, store records in a database or communicate with a bank. The scope is deliberately limited so that the rules and their outcomes can be inspected during a demonstration.

2. Need for the System and Proposed Solution

2.1 Issues in Manual Invoice Checking

Issue	What can happen	How the prototype responds
Calculation error	The total may not equal subtotal plus tax.	Recalculates the expected total using Decimal arithmetic.
Duplicate invoice	A bill may already exist in earlier records.	Compares the invoice number with existing sample invoices.
Payment mismatch	The amount recorded as paid differs from the billed amount.	Compares the payment amount with the invoice total.
Missing payment record	There is no payment entry available for reconciliation.	Marks the result incomplete rather than assuming wrongdoing.
Poor traceability	It may be difficult to explain how a decision was reached.	Displays reasons and a step-by-step audit log.

2.2 Proposed Solution

The solution converts invoice verification into a repeatable processing pipeline. When a user selects an invoice and starts processing, the orchestrator calls each agent in sequence. The agents share a common task: establish whether the supplied information satisfies the defined rules. Each check returns a clear outcome—PASS, FAIL or SKIPPED—and the Critic reviews the collected evidence before assigning the final status.

The workflow is designed to be conservative. A failed check prevents an invoice from being marked VERIFIED. Missing information is not automatically interpreted as fraud; instead, the system communicates that the available data is insufficient. Even when all checks pass, the output only indicates that the invoice is eligible for human review.

2.3 Why a Multi-Agent Structure?

The word 'agent' in this prototype refers to a software component with a specific responsibility. The design does not depend on autonomous language-model reasoning. Instead, it uses ordinary Python functions arranged to resemble a team: one component plans the work, one reads the data, one checks the rules and one reviews the evidence. This separation makes the process easier to demonstrate, test and maintain than placing all behaviour inside one large function.

2.4 Expected Benefits

- Consistency: the same rules are applied to each sample invoice.
- Explainability: the dashboard presents individual results rather than only a final label.
- Traceability: the audit log records the sequence of agent actions.
- Reduced repetitive work: routine comparisons are performed automatically within the prototype.
- Human oversight: the system does not execute payments or make an irreversible financial decision.

3. System Architecture

The architecture is organised into four logical layers: presentation, control, agents and data. Each layer has a clear purpose, while the orchestrator connects the interface to the processing functions.

3.1 Presentation Layer

The presentation layer is implemented with Streamlit in app.py. It allows the user to choose one of the available sample invoices, click the processing control and inspect the output. The interface separates information into views such as invoice details, validation checks and workflow/audit information. This structure helps the user move from the overall status to the evidence behind it.

3.2 Control Layer

The control layer is represented by the process_invoice() orchestrator. It controls the order in which agents run, applies a gate for empty or unreadable input, gathers their outputs and writes the audit log. Central coordination ensures that the agents follow the same processing sequence and that the final decision is based on the results collected during that run.

3.3 Agent Layer

Agent	Primary responsibility	Main output
Planner	Defines the ordered work required for verification.	Five-task plan.
Document	Reads the structured invoice data.	Six invoice fields.
Validation	Executes the five business checks.	PASS, FAIL or SKIPPED for each check.
Critic	Reviews the check results and assigns a status.	Final status and explanatory reasons.

3.4 Data Layer

The data layer contains six synthetic sample invoices, a set of existing invoice records for duplicate detection, and payment records for reconciliation. The records are held in memory for demonstration. As a result, the current implementation is simple to run, but its records are not a persistent source of truth and may not survive a restart.

3.5 Architecture Flow

User selects invoice → Streamlit interface → process_invoice() orchestrator → Planner → Document → empty-data gate → Validation → Critic → status, reasons, check results, reconciliation and audit log.

The sequence is intentionally linear. The Planner establishes what should happen, the Document agent supplies the fields, the gate stops processing if no usable information is available, the Validation agent checks the data, and the Critic produces the final interpretation. The interface then presents the collected outputs to the user.

4. Workflow and Agent Responsibilities

4.1 Step 1: Planning

The Planner creates a list of five tasks for the current invoice-processing run. Its purpose is to make the intended sequence explicit. In this simulation, planning is deterministic: the agent creates the defined task list rather than generating a new plan through a language model.

4.2 Step 2: Reading Invoice Fields

The Document agent reads six structured fields from the selected sample. These fields represent the invoice information needed by the subsequent checks, including identifying information, vendor details and monetary values. The present prototype expects structured data; it does not read text from a scanned page or photograph.

4.3 Step 3: Empty-Data Gate

Before validation proceeds, the orchestrator checks whether usable information was obtained. If the document stage produces no readable data, the workflow stops and returns INCOMPLETE. This gate avoids treating an empty input as though it had passed the business rules and helps prevent downstream errors.

4.4 Step 4: Validation

The Validation agent executes the five checks defined for the project. Each check produces a discrete result and a reason. A check may be marked SKIPPED when it cannot be evaluated reliably because required information is absent. Keeping the checks separate makes it possible to identify the exact issue rather than reporting a vague failure.

4.5 Step 5: Critic Review

The Critic examines the full set of validation results and decides the final status. It applies the project's decision rules: all five checks must pass for VERIFIED; a broken rule results in FLAGGED; and missing or unusable information leads to INCOMPLETE. The final response includes reasons so the user can understand the decision.

4.6 Output of a Processing Run

- Final status: VERIFIED, FLAGGED or INCOMPLETE.
- Reasons explaining why that status was assigned.
- Five individual validation results.
- Reconciliation details, including the invoice and payment amounts where available.
- An audit log recording agent activity in sequence.

This output is designed for review and demonstration. It provides evidence for a human decision, but it is not a payment instruction.

5. Validation Rules and Decision Logic

5.1 Required-Field Check

The first check confirms that the required invoice information is present. Missing fields can prevent later checks from being meaningful. For example, without an invoice identifier the system cannot reliably perform a duplicate comparison, and without monetary values it cannot establish whether the arithmetic is consistent.

5.2 Total Calculation

The second check verifies the relationship between subtotal, tax and total. The expected relationship is:

$$\text{Expected Total} = \text{Subtotal} + \text{Tax}$$

The prototype uses Python's Decimal type for monetary arithmetic rather than relying on binary floating-point values. This is appropriate for a financial demonstration because it helps avoid unexpected representation differences in decimal amounts. The check passes only when the supplied total agrees with the computed value under the implemented rule.

5.3 Duplicate Invoice Check

The third check compares the current invoice number with the available earlier invoice records. If the same identifier already appears, the system flags a possible duplicate. This is a warning that requires review; it does not independently prove that an invoice is fraudulent, because legitimate business situations can sometimes require contextual investigation.

5.4 Payment Record Check

The fourth check verifies whether a payment record exists for the invoice. If no matching record is available, the system marks the case INCOMPLETE because reconciliation cannot be completed from the supplied information. The project explicitly avoids treating the absence of a payment record as proof of fraud.

5.5 Payment Amount Check

The fifth check compares the recorded payment amount with the invoice total. A difference indicates a payment mismatch and causes the invoice to be flagged. The output can show the size of the difference to help a reviewer understand the issue.

5.6 Meaning of the Three Check Results

Result	Meaning
PASS	The available information satisfies the rule.
FAIL	The rule has identified a mismatch or violation.
SKIPPED	The check could not be evaluated because the necessary information was unavailable.

6. Final Statuses and Human Oversight

6.1 VERIFIED

VERIFIED is returned only when all five validation checks pass. It means that the sample data satisfies the rules implemented in this prototype and is eligible for human review. It does not mean that the system has paid the vendor, authorised a transfer or established that every real-world risk has been eliminated.

6.2 FLAGGED

FLAGGED is used when a validation rule fails. Typical examples include a total that does not match subtotal plus tax, a repeated invoice number or a payment amount that differs from the invoice total. The status directs attention to an issue that should be examined before the invoice progresses.

6.3 INCOMPLETE

INCOMPLETE indicates that the system does not have enough usable information to reach a reliable conclusion. Examples include a blank required field, an invalid number or the absence of a payment record. This status communicates uncertainty and avoids forcing incomplete information into a pass/fail interpretation.

6.4 Decision Principles

- A failed check can never produce VERIFIED.
- All five checks must pass for VERIFIED.
- Missing information should be reported as incomplete rather than silently accepted.
- An absent payment record is not, by itself, classified as fraud.
- The final outcome is a recommendation for human review, not an automatic financial action.

6.5 Why Human Control Matters

Financial workflows have consequences for organisations and vendors. A false approval can allow an incorrect payment to proceed, while an incorrect flag can delay a legitimate settlement. The project therefore keeps the final authority with a human reviewer. The dashboard provides an organised summary of evidence, leaving the reviewer responsible for resolving exceptions and deciding what should happen next.

6.6 Auditability

The audit log records agent activity in order and includes the Critic’s decision. Along with per-check explanations, this creates a basic trail for understanding how the result was produced. The log is intended for demonstration and is not equivalent to a secured, tamper-resistant audit system used in a production financial environment.

7. Implementation and Demonstration

7.1 Technologies Used

Technology	Use in the project
Python 3	Implements the processing functions and rule-based agents.
Streamlit	Provides the browser-based dashboard and invoice selection.
Decimal	Supports exact decimal arithmetic for monetary checks.
Mermaid	Used to represent the architecture or workflow diagram.

7.2 Running the Prototype

1. Open a terminal in the project directory containing app.py.
2. Start the dashboard with the command: streamlit run app.py
3. Open the local address shown by Streamlit; the presentation identifies localhost:8501 as the usual dashboard address.
4. Choose an invoice from the sidebar and select Process Invoice.
5. Review the invoice fields, five check results, reconciliation details and audit log.

7.3 Sample Invoices

Invoice	Scenario represented	Expected outcome
INV101	Correct invoice and matching payment.	VERIFIED
INV102	Payment of ₹20,000 against a total of ₹23,600.	FLAGGED
INV103	Invoice number already appears in earlier records.	FLAGGED
INV104	Incorrect invoice total.	FLAGGED
INV105	Vendor field is missing.	INCOMPLETE
INV106	No payment record is available.	INCOMPLETE

7.4 Demonstration Sequence

The recommended demonstration begins with INV101 to show the successful path. The dashboard should show a VERIFIED result, five passing checks and a matching reconciliation. In the example, a subtotal of ₹10,000 plus tax of ₹1,800 equals the invoice total of ₹11,800. The displayed payment record is dated 2026-10-10.

Next, INV102 demonstrates an amount mismatch: ₹20,000 has been paid against an invoice total of ₹23,600, leaving a difference of ₹3,600. INV103 demonstrates duplicate detection. Finally, INV106 demonstrates how a missing payment record leads to INCOMPLETE rather than an allegation of fraud. INV104 and INV105 provide additional examples for arithmetic failure and missing required data.

8. Testing and Results

8.1 Testing Approach

The presentation reports nine tests, all passing. The tests are intended to check whether the rules produce the expected business outcomes and whether the program handles problematic inputs without crashing. The supplied presentation does not provide a detailed test-run transcript or code-level coverage report, so the results below summarise the stated test scope rather than claiming additional measurements.

8.2 Reported Test Coverage

Test area	Purpose	Expected behaviour
Six business cases	Cover the required invoice scenarios.	Statuses reflect the defined rules.
Empty input	Check behaviour when no usable data is read.	Processing stops safely and returns INCOMPLETE.
Invalid number	Check malformed monetary input.	The application handles it without crashing.
Failed-check safeguard	Protect the final decision rule.	A failed check cannot result in VERIFIED.

8.3 Observed Demonstration Outcomes

The sample scenarios illustrate three different decision outcomes. INV101 shows that a fully consistent invoice can reach VERIFIED. INV102 and INV103 show that a payment mismatch and a possible duplicate are surfaced as FLAGGED. INV106 shows that missing reconciliation data produces INCOMPLETE. INV104 and INV105 broaden the demonstration by testing a wrong total and a missing vendor field.

8.4 Interpretation

The results support the intended behaviour of the rule-based prototype: it separates successful validation from explicit rule failures and insufficient information. Its value is not a claim that it can detect every invoice risk. Rather, it demonstrates that a small set of transparent checks can be coordinated, reported consistently and reviewed through a user interface.

8.5 What Has Not Been Evaluated

No performance benchmark, large-scale dataset evaluation, financial fraud detection accuracy, security assessment or comparison with a production invoice platform is reported in the presentation. Since the records are synthetic and held in memory, the prototype cannot establish how well it would perform on real invoices, large volumes of records or inconsistent data from multiple business systems.

9. Demonstration of the Invoice Processing Dashboard

The following demonstration screenshots are taken from the running Streamlit prototype at localhost:8501. They document the interface and the output produced for the selected sample invoice. The application uses synthetic demonstration data and does not make real payments.

9.1 Dashboard before processing

Initially, the dashboard asks the user to select an invoice from the sidebar and click Process Invoice. The first screen shows the main interface before any result has been generated.

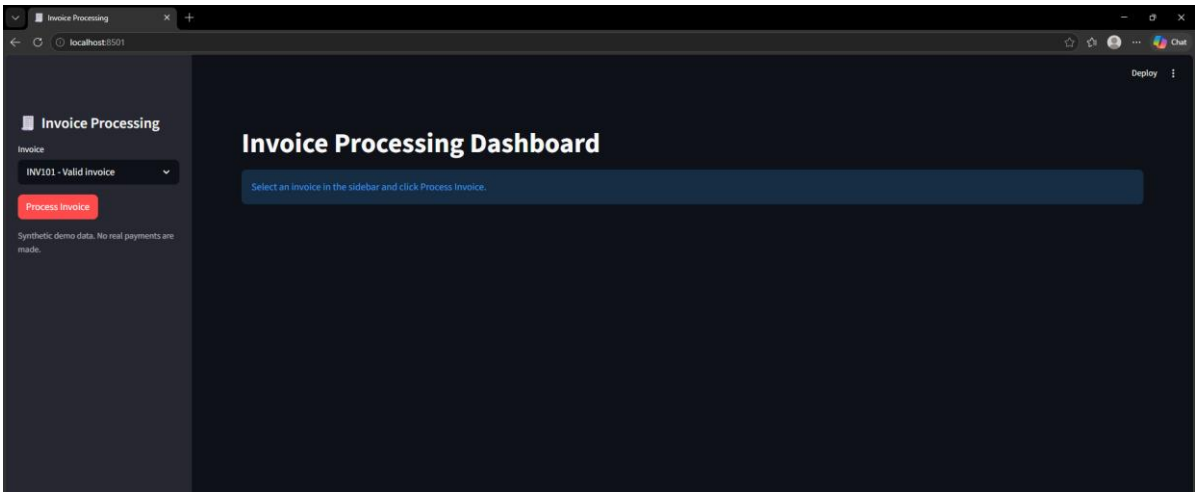


Figure 1. Invoice Processing dashboard before processing an invoice.

9.2 Selecting a sample invoice

The sidebar provides six synthetic cases: INV101 (valid invoice), INV102 (payment mismatch), INV103 (duplicate invoice), INV104 (wrong total), INV105 (missing vendor) and INV106 (no payment record). Selecting a case allows the same workflow to be demonstrated against different conditions.

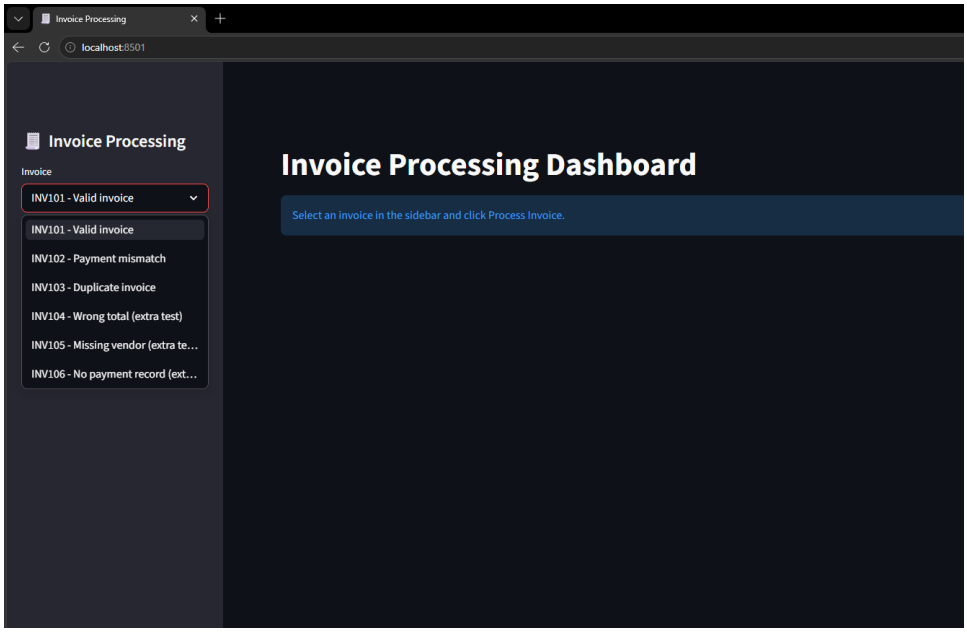


Figure 2. Sample invoice options in the sidebar.

9.3 Successful case: INV101

After INV101 is processed, the dashboard displays a green VERIFIED banner and explains that the invoice is eligible for human approval review. The invoice total and payment recorded are both ₹11,800. The invoice fields show a subtotal of ₹10,000 and tax of ₹1,800, which add up to the stated total.

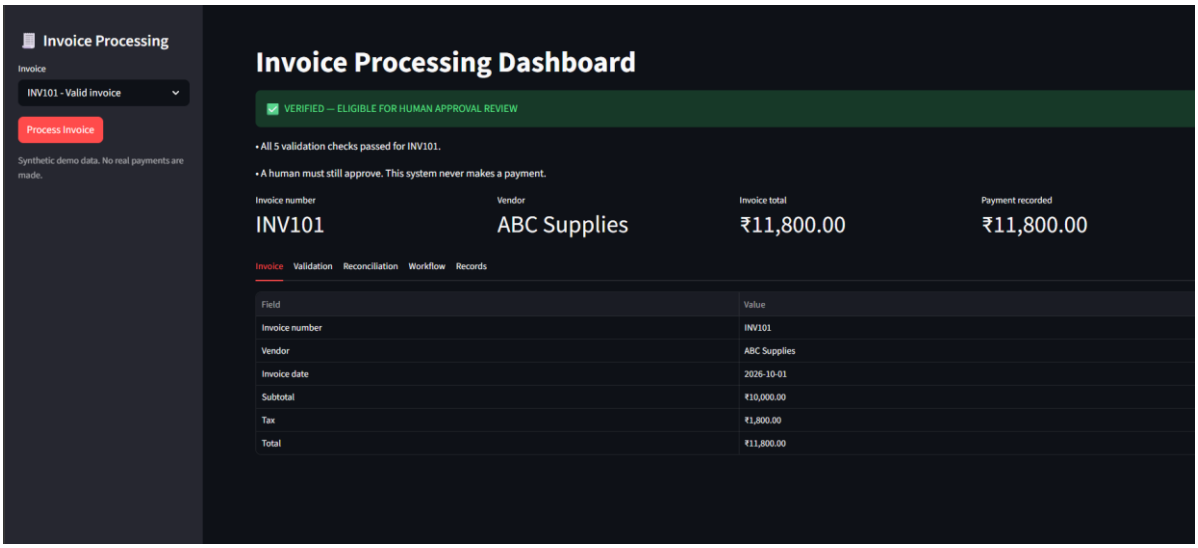


Figure 3. INV101 result and invoice details.

9.4 Validation results

The Validation tab displays the five checks and their individual outcomes. For INV101, all five are marked Pass: required fields are present; subtotal plus tax equals the total; the invoice number is not a duplicate; a payment record exists; and the payment amount matches the invoice total. The payment record shown is dated 2026-10-10.

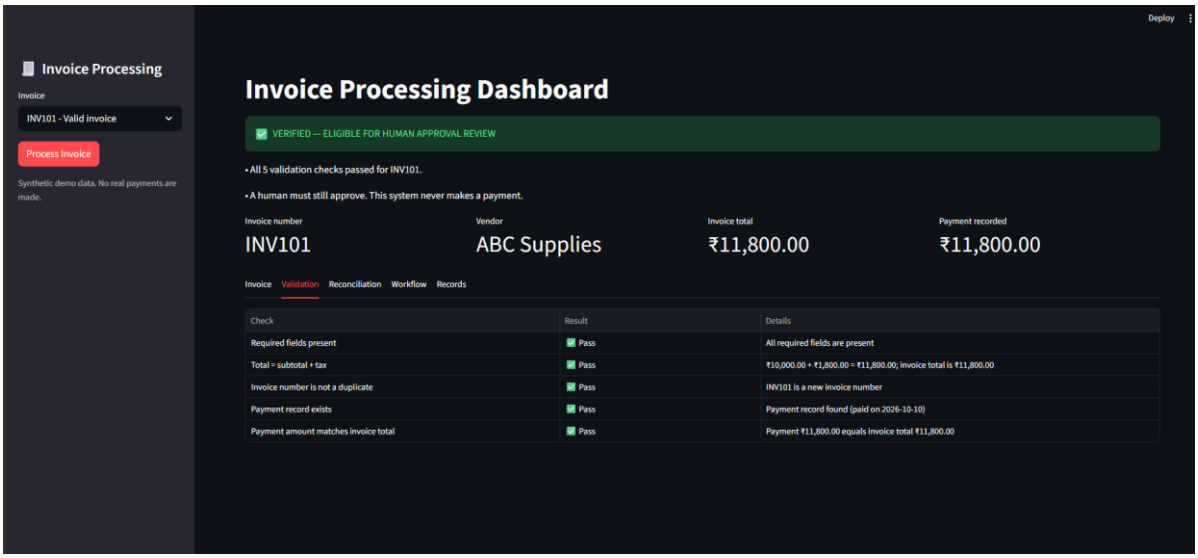


Figure 4. Validation tab showing all five checks passing.

9.5 Workflow and audit log

The Workflow tab shows the five planned tasks alongside the audit log. The log records the Planner, Document Processing, Validation and Critic agent actions in sequence. It also records the final VERIFIED decision. The audit trail helps the user understand how the status was reached.

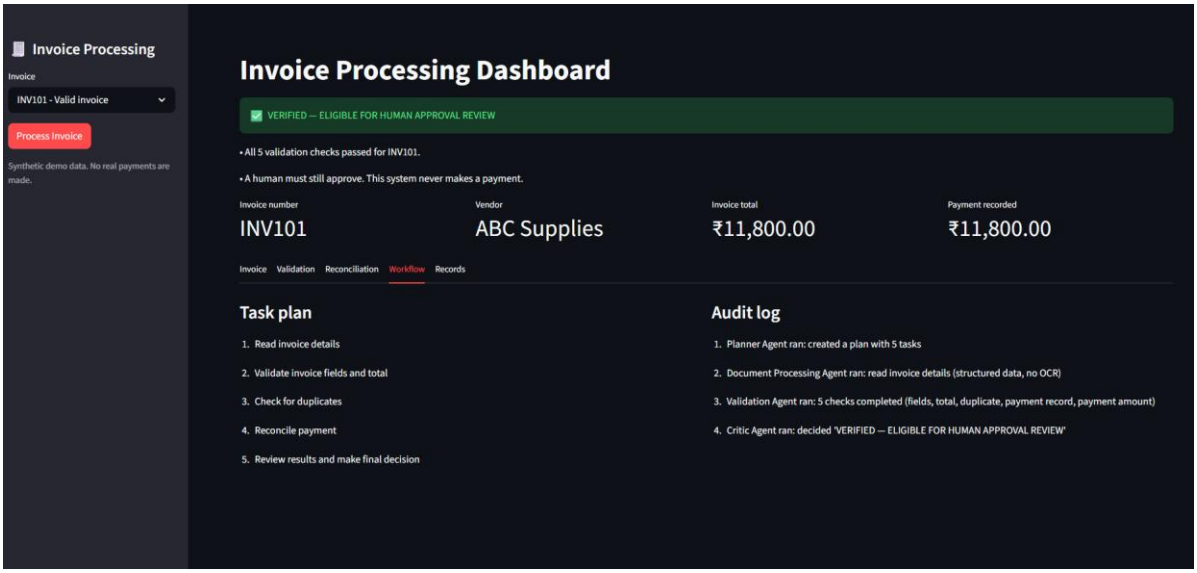


Figure 5. Workflow plan and audit log for INV101.

9.6 Additional scenarios to demonstrate

The remaining test cases should be processed to illustrate how the system responds when a rule fails or information is missing. INV102 should show a payment mismatch between ₹20,000 paid and an invoice total of ₹23,600, a difference of ₹3,600. INV103 demonstrates duplicate detection. INV104 demonstrates an incorrect total. INV105 demonstrates a missing vendor field. INV106 demonstrates the absence of a payment record and should be treated as INCOMPLETE rather than as proof of fraud.

Note: The screenshots supplied show the initial dashboard, invoice selection, successful INV101 processing, its validation checks, and its workflow/audit log. Screenshots for the other invoice outcomes were not supplied, so their behaviour is described from the project presentation rather than illustrated here.

9. Limitations and Future Scope

9.1 Current Limitations

- Rule-based processing: the agents follow predefined Python logic and do not learn from examples.
- Structured input only: the current system does not use OCR to read scanned invoices or photographs.
- Synthetic data: the six sample invoices are for demonstration and do not represent a live business dataset.
- In-memory storage: invoice and payment records are not maintained in a persistent database.
- No external integrations: the prototype does not connect to bank APIs or initiate actual payments.
- Limited validation scope: only the five defined checks are performed; other business, tax, vendor or compliance rules are outside the stated scope.

9.2 Future Enhancements

Optical Character Recognition

An OCR component could be added to extract text from scanned invoices. Extracted values would still need validation, because OCR can misread characters, amounts and invoice identifiers.

Language-Model-Based Agents

A future version could use language-model agents to interpret varied invoice layouts, explain unusual cases or help a reviewer investigate exceptions. Any such component should be evaluated carefully and should not override deterministic financial checks without controls.

Database Integration

A persistent database could store invoices, payment records, processing outcomes and audit events. It would make it possible to retain history across restarts and support more realistic duplicate detection.

Banking and Enterprise APIs

Secure integration with authorised payment or enterprise systems could provide reliable payment data. Such integrations would require authentication, access control, error handling, privacy protections and careful testing. Payment execution should remain subject to explicit organisational approval.

Additional Controls

Further development could introduce role-based access, stronger audit-log protection, configurable business rules, exception queues and reporting on repeated errors. These are proposed extensions and are not features claimed for the current implementation.

10. Conclusion

This project demonstrates a rule-based multi-agent approach to banking and financial invoice verification. It organises a common task into four specialised components: the Planner prepares the task sequence, the Document agent reads structured fields, the Validation agent runs five checks, and the Critic reviews the evidence to produce the final status. A central orchestrator coordinates these stages and records the workflow.

The prototype focuses on five checks: required-field completeness, arithmetic consistency between subtotal and tax, duplicate invoice detection, payment-record availability and payment-amount matching. The use of Decimal arithmetic supports precise monetary comparisons. Through a Streamlit dashboard, users can inspect the invoice details, check outcomes, reconciliation information and audit log rather than relying on an unexplained final label.

The three statuses—VERIFIED, FLAGGED and INCOMPLETE—communicate different conditions. VERIFIED means that all implemented checks passed and the invoice is ready for human review. FLAGGED identifies a rule failure. INCOMPLETE indicates that essential information is missing or unusable. This distinction is important because the absence of a payment record should not automatically be interpreted as fraudulent activity.

The project remains a limited simulation. It relies on fixed rules, structured synthetic records and in-memory data; it does not use OCR, a language model, a persistent database or live payment connections. The reported nine tests cover the stated business scenarios and selected robustness safeguards, but broader real-world evaluation would be needed before operational use.

Overall, the prototype illustrates how a coordinated agent-style workflow can make routine invoice checks more consistent and easier to explain. Its most important design principle is that automation supports the reviewer rather than replacing human judgement or taking control of financial transactions. Future work can expand data extraction, persistence and integrations while retaining explicit validation rules and human approval.

References / Basis of Preparation

1. Project presentation supplied by the project author: “Banking & Financial Invoice Processing using Agentic AI (rule-based multi-agent simulation).” The report follows the system scope, terminology, sample scenarios and reported testing described in that presentation.

2. Python 3 standard library documentation, including Decimal, for general technical background on decimal arithmetic.

3. Streamlit documentation, for general background on creating interactive Python web applications.

Note: Items 2 and 3 are suggested technical references for the final submission. If your institution requires a formal reference style, add the exact documentation URLs and access dates used during project development.

Appendix A: Key Terms

Term	Meaning in this project
Agent	A software component assigned a specific task.
Orchestrator	The function that controls the order of processing.
Reconciliation	Comparing invoice values with payment records.
Audit log	A chronological record of actions and decisions.
Human-in-the-loop	A process where a person retains decision authority.